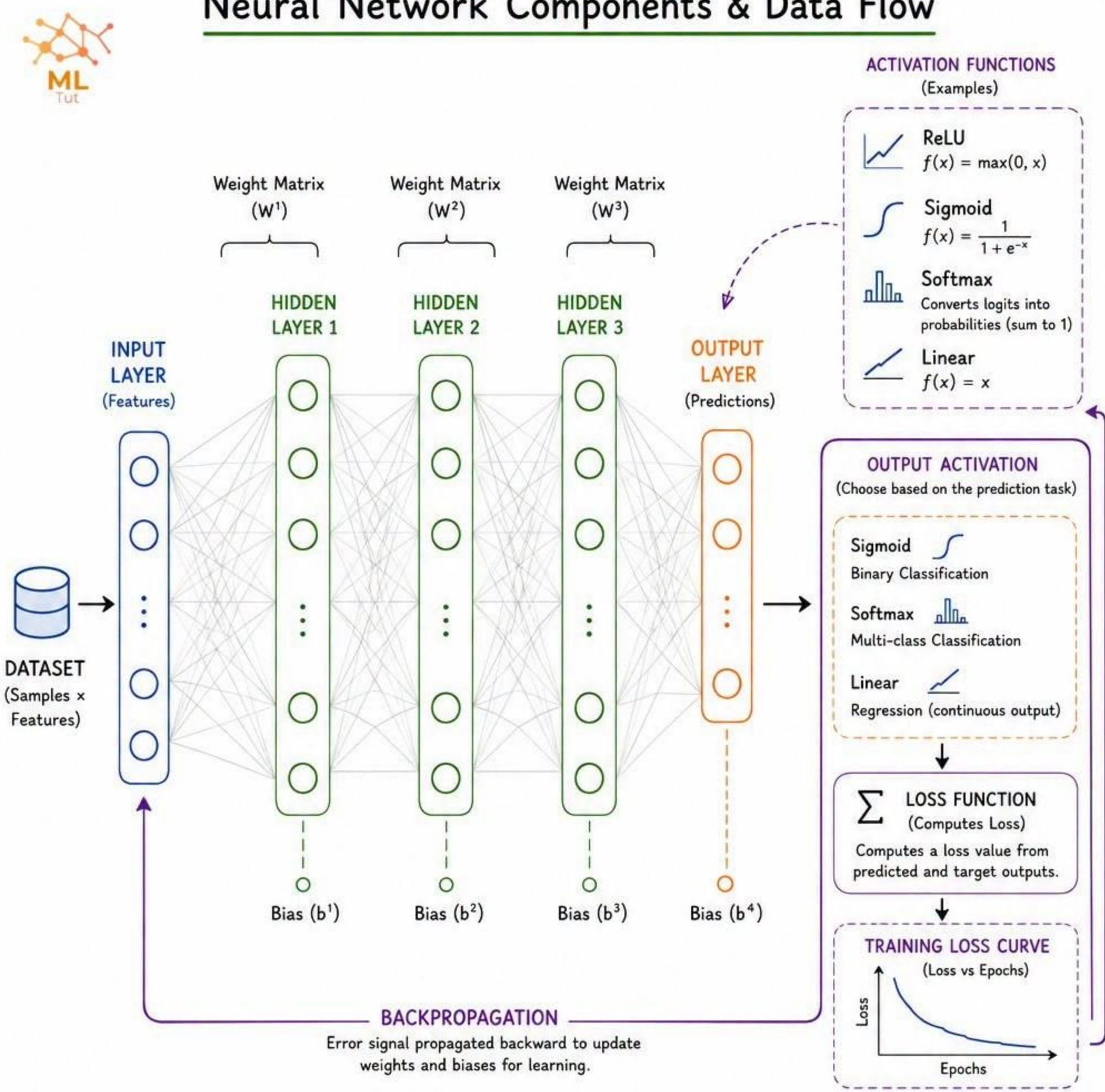


DEEP LEARNING ANATOMY STRUCTURE

Neural Network Components & Data Flow



COMPONENTS

Input Layer

Receives feature vectors from the dataset.

Hidden Layers

Learn feature representations using weights, biases, and activation functions.

Output Layer

Produces predictions for the given task.

FLOW SUMMARY

1

Input data flows forward through the network.

2

Each layer learns a weight matrix (W) and a bias vector (b).

3

Activation functions introduce non-linearity and help the network learn complex patterns.

4

The output layer generates predictions.

5

The loss function computes the training loss.

6

Gradients are backpropagated and weights and biases are updated to minimize the loss.

COMMON ACTIVATIONS

ReLU

$f(x) = \max(0, x)$

Most common in hidden layers.

Sigmoid

$f(x) = \frac{1}{1 + e^{-x}}$

Outputs between 0 and 1.

Softmax

Converts logits into class probabilities (sum to 1).

Used for multi-class classification.

Linear

$f(x) = x$

Used for regression (continuous output).

TYPES OF NEURAL NETWORKS

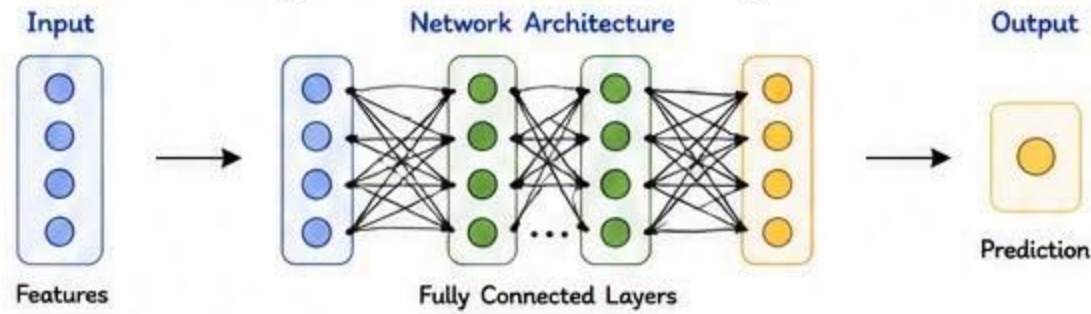
Different types of neural networks solve different kinds of problems.

Choose the right architecture for the right task!

1. FEEDFORWARD NEURAL NETWORK (FNN)

- Simplest type of neural network
- Information moves in one direction: input → hidden → output
- Used for basic prediction problems

Example of FNN:
Multilayer Perceptron (MLP)

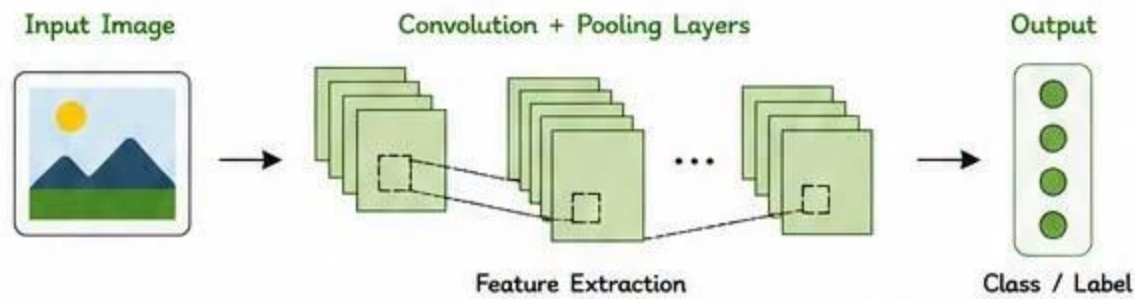


Example

Predict house prices based on size, location, number of rooms.

2. CONVOLUTIONAL NEURAL NETWORK (CNN)

- Designed for grid-like data such as images
- Uses convolutional filters to detect local patterns
- Excellent for image processing

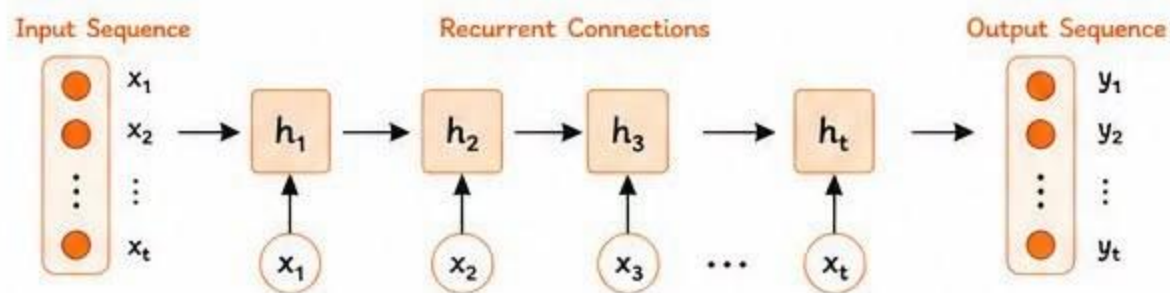


Example

Image classification (e.g., cat, dog)

3. RECURRENT NEURAL NETWORK (RNN)

- Designed for sequential data
- Uses recurrent connections to process previous information
- Used for sequence and time-series data

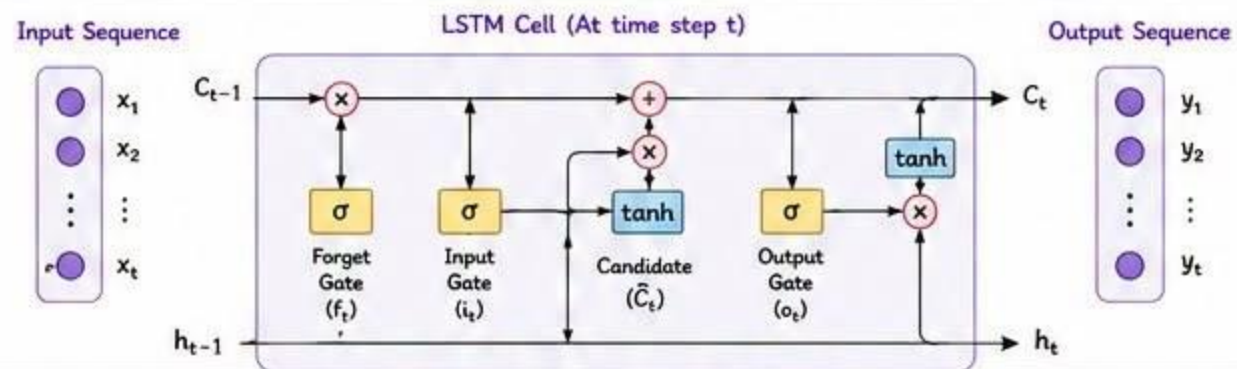


Example

Sequence modeling, time-series forecasting

4. LONG SHORT-TERM MEMORY (LSTM)

- Advanced RNN with memory cells
- Uses gates to control information flow
- Helps reduce the vanishing-gradient problem in RNNs

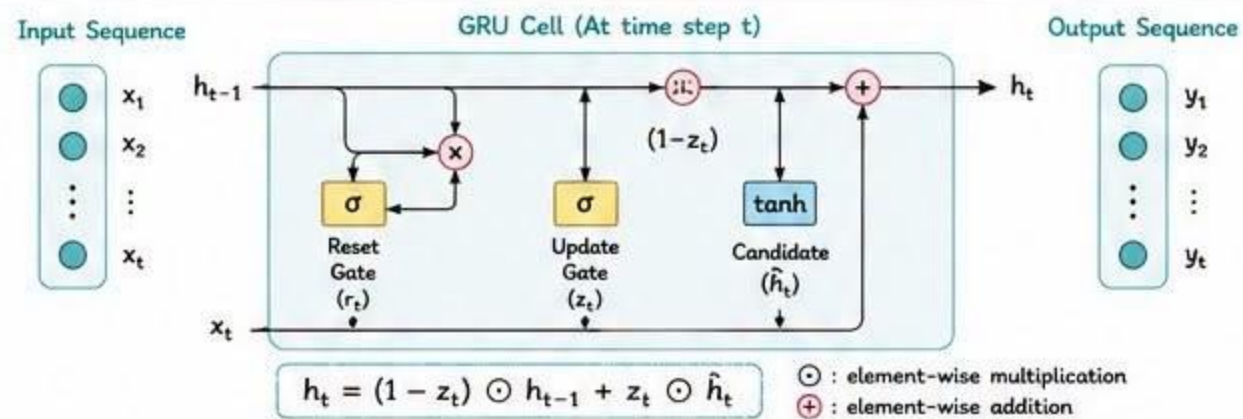


Example

Machine translation, speech recognition, time-series forecasting

5. GATED RECURRENT UNIT (GRU)

- Gated RNN with a simpler architecture than LSTM
- Uses update and reset gates to control information flow
- Typically has fewer parameters than LSTM

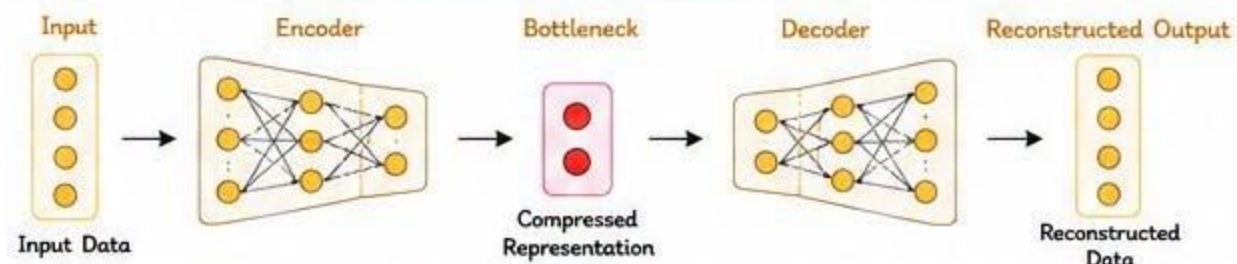


Example

Text generation, sentiment analysis, real-time applications

6. AUTOENCODER (AE)

- Self-supervised neural network
- Learns to compress input and reconstruct it back
- Used for dimensionality reduction and anomaly detection

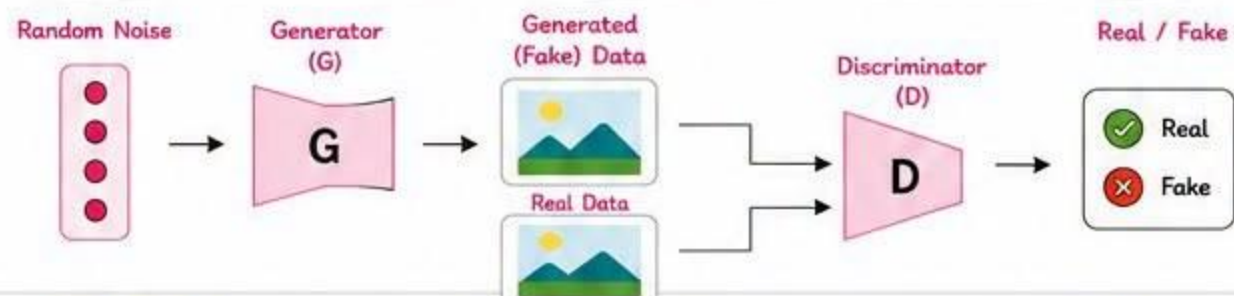


Example

Anomaly detection, image compression

7. GENERATIVE ADVERSARIAL NETWORK (GAN)

- Consists of two networks: Generator and Discriminator
- Learns to generate realistic new data

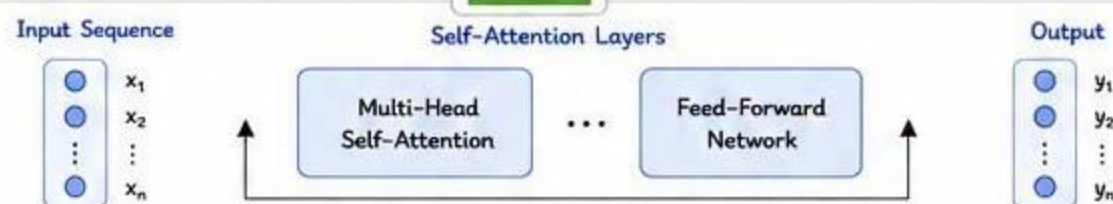


Example

Image generation, deepfakes, creative AI

8. TRANSFORMER NETWORK

- Uses self-attention mechanism
- Captures long-range dependencies using attention
- Foundation of many modern NLP and multimodal models



Example

ChatGPT, BERT, text summarization, code generation

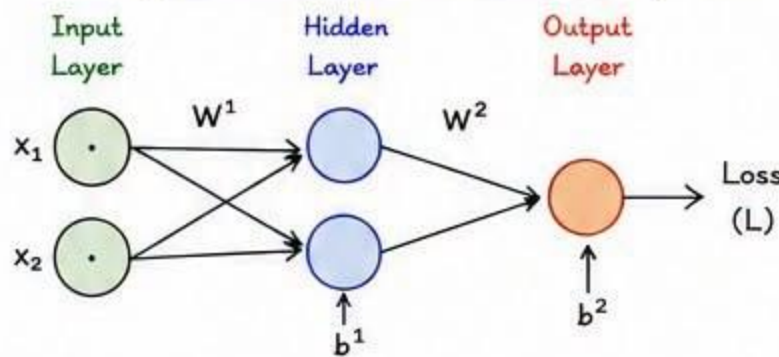
BACKPROPAGATION

Using Python

What is Backpropagation?

Backpropagation is the algorithm used to train neural networks. It computes the gradients of the loss with respect to model parameters (weights and biases) by propagating gradients backward through the network, layer by layer, and updates the parameters using gradient descent.

1. Neural Network Overview



W^1 : Weights (input $\rightarrow$ hidden)
 W^2 : Weights (hidden $\rightarrow$ output)
 b^1 : Bias (hidden layer)
 b^2 : Bias (output layer)
 $\hat{y}$: Predicted output
 L : Loss

2. Forward Pass

$$Z^1 = XW^1 + b^1$$

$$A^1 = \sigma(Z^1)$$

$$Z^2 = A^1W^2 + b^2$$

$$\hat{y} = \sigma(Z^2)$$

σ : Activation function (Sigmoid)
 X : Input features

3. Loss Function

We use Squared Error Loss

$$L = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Where,

y = True output
 $\hat{y}$ = Predicted output
 m = Number of samples

4. Backward Pass (Backpropagation)

Compute gradients using the chain rule.

Output Layer:

$$dZ^2 = (\hat{y} - y) * \sigma'(Z^2)$$

$$dW^2 = A^{1T} dZ^2 / m$$

$$db^2 = \sum dZ^2 / m$$

Hidden Layer:

$$dA^1 = dZ^2 W^{2T}$$

$$dZ^1 = dA^1 * \sigma'(Z^1)$$

$$dW^1 = X^T dZ^1 / m$$

$$db^1 = \sum dZ^1 / m$$

$$\sigma'(z) = \sigma(z) * (1 - \sigma(z))$$

5. Update Weights (Gradient Descent)

Update parameters using learning rate (α).

$$W^2 = W^2 - \alpha dW^2$$

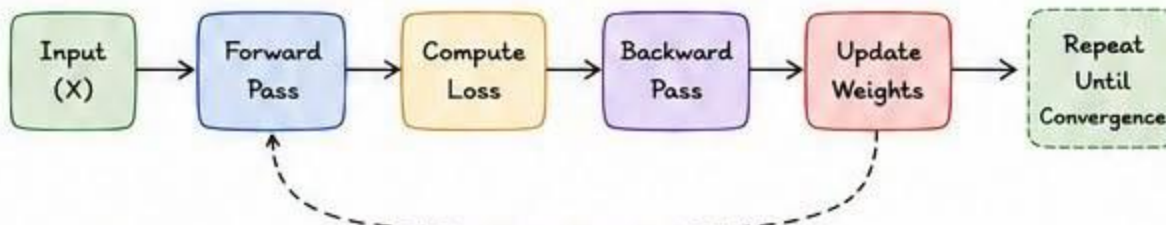
$$b^2 = b^2 - \alpha db^2$$

$$W^1 = W^1 - \alpha dW^1$$

$$b^1 = b^1 - \alpha db^1$$

α : Learning rate

6. Backpropagation Flow



7. Backpropagation in Python (NumPy)

```
import numpy as np

# Sigmoid and its derivative
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    s = sigmoid(z)
    return s * (1 - s)

# Hyperparameters
np.random.seed(42)
X = np.array([[0,0], [0,1], [1,0], [1,1]]) # XOR inputs
y = np.array([[0], [1], [1], [0]]) # XOR outputs

# Initialize weights and biases
W1 = np.random.randn(2,2) * 0.5
b1 = np.zeros((1,2))
W2 = np.random.randn(2,1) * 0.5
b2 = np.zeros((1,1))

learning_rate = 1.0
epochs = 10000

for epoch in range(epochs):
    # ---- Forward Pass ----
    Z1 = X.dot(W1) + b1
    A1 = sigmoid(Z1)
    Z2 = A1.dot(W2) + b2
    y_hat = sigmoid(Z2)

    # ---- Loss (Squared Error) ----
    loss = 0.5 * np.mean((Y - y_hat) ** 2)

    # ---- Backward Pass ----
    dZ2 = (y_hat - Y) * sigmoid_derivative(Z2)
    dW2 = A1.T.dot(dZ2) / len(X)
    db2 = np.sum(dZ2, axis=0, keepdims=True) / len(X)

    dA1 = dZ2.dot(W2.T)
    dZ1 = dA1 * sigmoid_derivative(Z1)
    dW1 = X.T.dot(dZ1) / len(X)
    db1 = np.sum(dZ1, axis=0, keepdims=True) / len(X)

    # ---- Update Weights ----
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    if epoch % 1000 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.6f}")

# Predictions after training
y_hat = (y_hat >= 0.5).astype(int)
print("\nPredictions after training:", y_hat)
```

8. Example Output

```
Epoch 0, Loss: 0.125078
Epoch 1000, Loss: 0.124999
Epoch 2000, Loss: 0.003147
Epoch 3000, Loss: 0.000727
Epoch 4000, Loss: 0.000284
Epoch 5000, Loss: 0.000132
Epoch 6000, Loss: 0.000070
Epoch 7000, Loss: 0.000040
Epoch 8000, Loss: 0.000025
Epoch 9000, Loss: 0.000017
...
Predicted classes after training:
[[0]
 [1]
 [1]
 [0]]
```

9. Key Points

- Forward pass computes predictions.
- Loss measures how far predictions are from actual values.
- Backpropagation computes gradients layer by layer (backward).
- Weights are updated to minimize loss.
- Repeat until the model converges.

Common Activation Functions

- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$
- Tanh: $\tanh(z)$
- ReLU: $\max(0, z)$

Backpropagation is the brain behind learning in Neural Networks! 🧠

Practice. Experiment. Improve.

Helpful Libraries

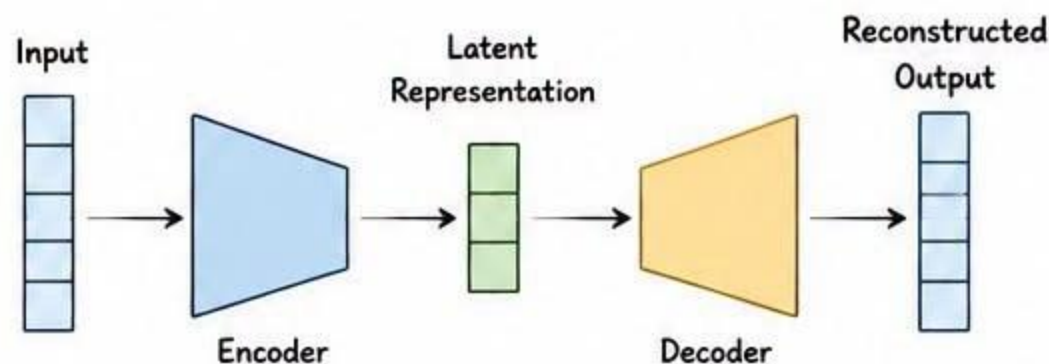
NumPy • Matplotlib • PyTorch • TensorFlow

ENCODER ARCHITECTURES IN MACHINE LEARNING

Encoders convert input data into meaningful representations that capture important patterns and features.

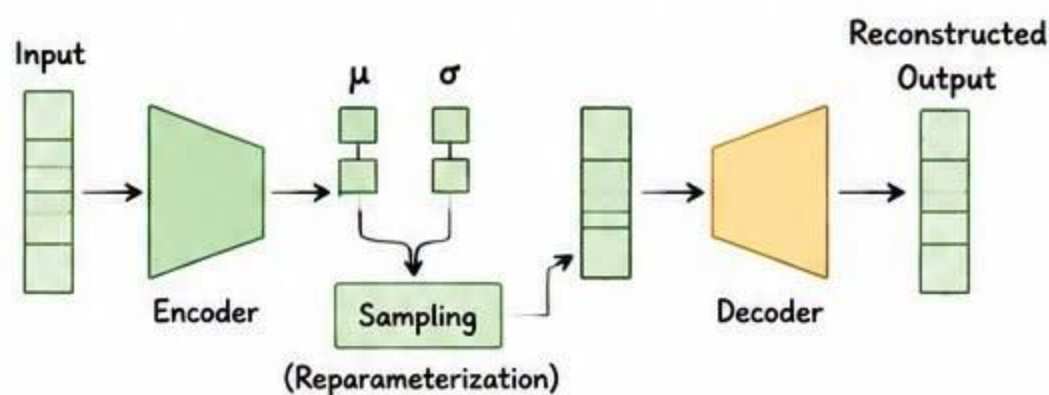
1. AUTOENCODER

- Unsupervised learning
- Learns compressed representation
- Encoder-decoder structure
- Used for dimensionality reduction, denoising, anomaly detection



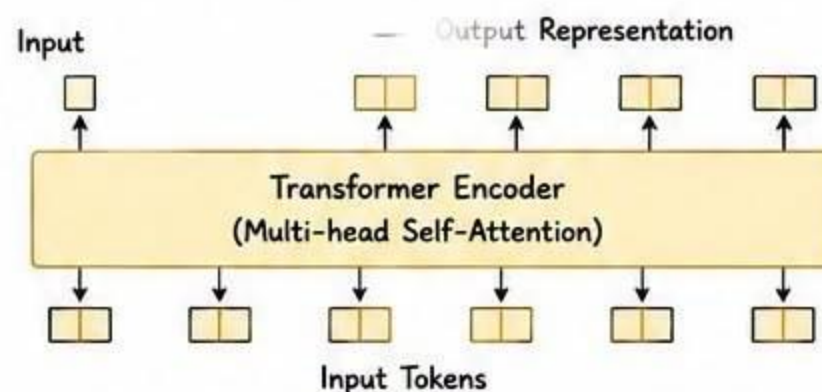
2. VARIATIONAL AUTOENCODER (VAE)

- Probabilistic extension of autoencoder
- Learns distribution in latent space
- Used for generative modeling, data synthesis



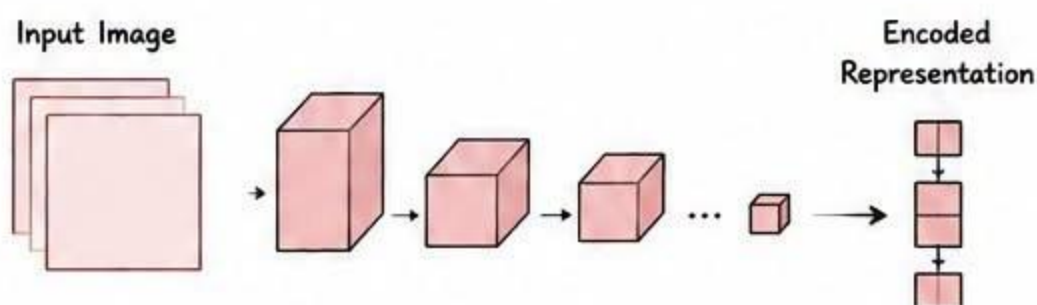
3. TRANSFORMER ENCODER

- Based on self-attention mechanism
- Captures long-range dependencies
- Widely used in NLP, vision, and multimodal tasks



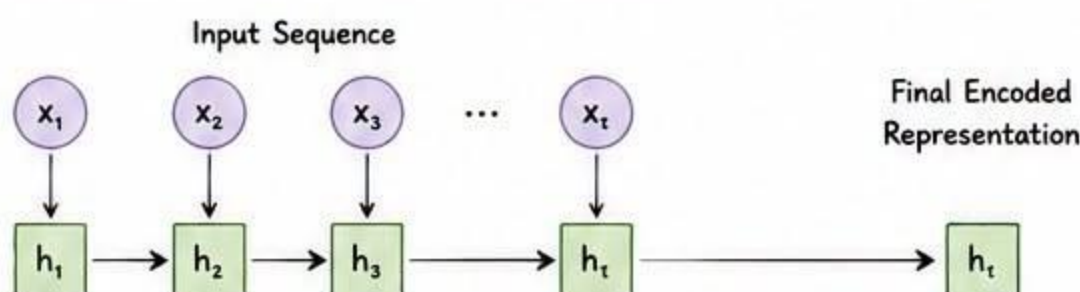
4. CNN ENCODER

- Uses convolutional layers
- Captures spatial hierarchies
- Common in image and video understanding



5. RNN / LSTM ENCODER

- Processes sequential data
- Maintains hidden state
- Used in NLP, time series, speech, etc.



Key Takeaway

Encoders transform raw data into rich representations. The right encoder depends on your data type and task.

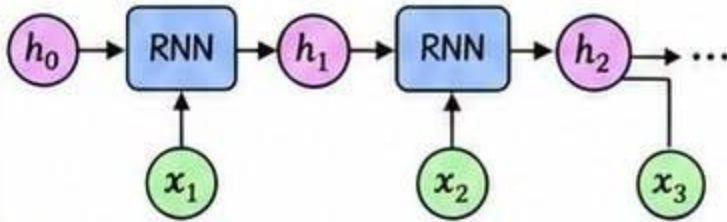
- Images → CNN Encoder
- Text / Sequence → Transformer or RNN
- Unsupervised Representation → Autoencoder / VAE

RNN vs LSTM vs GRU

Understand the differences, architectures and when to use each

1. What is RNN?

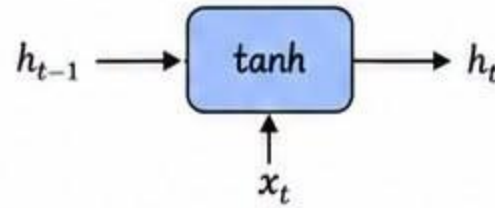
Recurrent Neural Network (RNN) is designed to handle sequential data by maintaining a hidden state (memory) from previous steps.



- Uses the same weights at each time step.
- Captures short-term dependencies well.

2. RNN Architecture

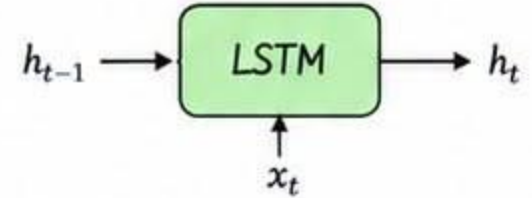
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$



- Simple architecture.
- Single hidden state.
- Can struggle with long-term dependencies (vanishing gradient problem).

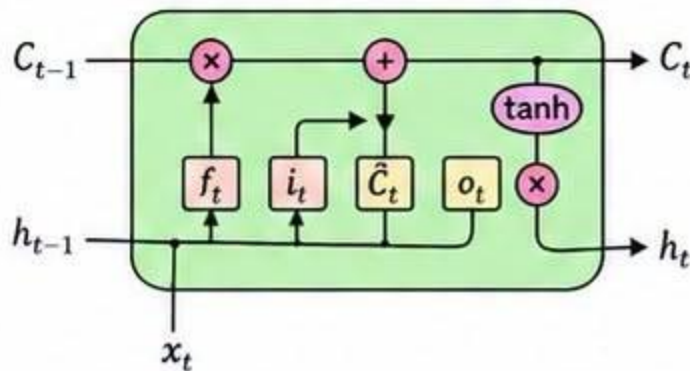
3. What is LSTM?

Long Short-Term Memory (LSTM) is an advanced RNN that uses gates to control the flow of information and can capture long-term dependencies.



- Uses input, forget and output gates.
- Can remember information for longer periods.
- Works well for complex sequential data.

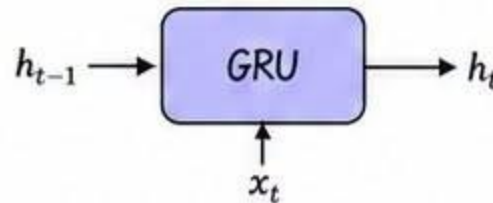
4. LSTM Architecture



- Has three gates: Forget (f_t), Input (i_t), Output (o_t).
- Maintains a cell state (C_t) for long-term memory.

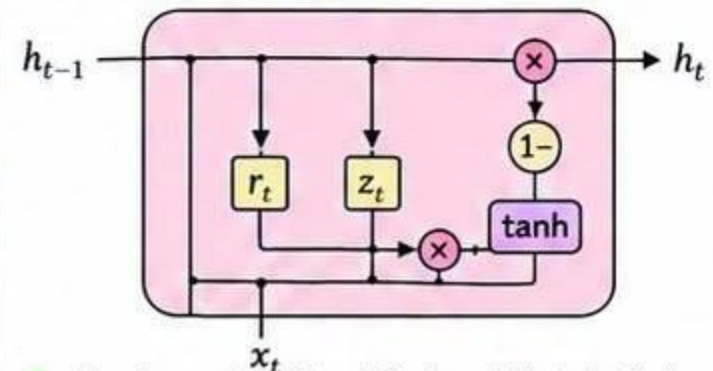
5. What is GRU?

Gated Recurrent Unit (GRU) is a simplified version of LSTM with fewer gates, but still effective in capturing long-term dependencies.



- Uses reset and update gates.
- No separate cell state (simpler structure).
- Often trains faster than LSTM.

6. GRU Architecture



- Has two gates: Reset (r_t) and Update (z_t).
- Uses a single hidden state instead of separate cell and hidden states.
- Fewer parameters than LSTM.

7. Key Differences

Feature	RNN	LSTM	GRU
Gates	No	3 (Input, Forget, Output)	2 (Reset, Update)
Memory	Hidden state (h_t)	Cell state (C_t) + hidden state	Single hidden state (h_t)
Handles long dependencies	Poor	Excellent	Good
Complexity	Simple	More complex	Simpler than LSTM
Training time	Fast	Slower	Faster than LSTM
Parameters	Few	More	Fewer than LSTM

8. When to Use?

Use RNN if:

- Simple tasks
- Short sequences
- When computational resources are limited

Use LSTM if:

- Long-term dependencies
- Complex sequences
- Long sequence modeling is important

Use GRU if:

- Need a simpler model than LSTM
- Want faster training
- Want fewer parameters than LSTM

Rule of Thumb:

Start with GRU → if performance is not enough, try LSTM → use simple RNN for basic tasks.

9. Code Example (PyTorch)

```
import torch
import torch.nn as nn

# RNN
rnn = nn.RNN(input_size=10, hidden_size=20,
              num_layers=1, batch_first=True)

# LSTM
lstm = nn.LSTM(input_size=10, hidden_size=20,
               num_layers=1, batch_first=True)

# GRU
gru = nn.GRU(input_size=10, hidden_size=20,
              num_layers=1, batch_first=True)
```

10. Real-World Applications

RNN

- Text prediction
- Simple NLP tasks
- Time series forecasting

LSTM

- Long-sequence modeling
- Time series prediction
- Legacy NLP/speech systems
- Where long-term dependencies matter

GRU

- Text generation
- Chatbots
- Time series forecasting
- When faster training is needed

11. Performance Comparison

Aspect	RNN	LSTM	GRU
Training speed (generally)	Fastest	Often slower than GRU	Often faster than LSTM
Number of parameters (same input/hidden size)	Lowest	Highest	Moderate
Handles long-term dependencies	Weak	Strong	Good
Model complexity	Simple	Most complex	Moderate

12. Key Takeaways

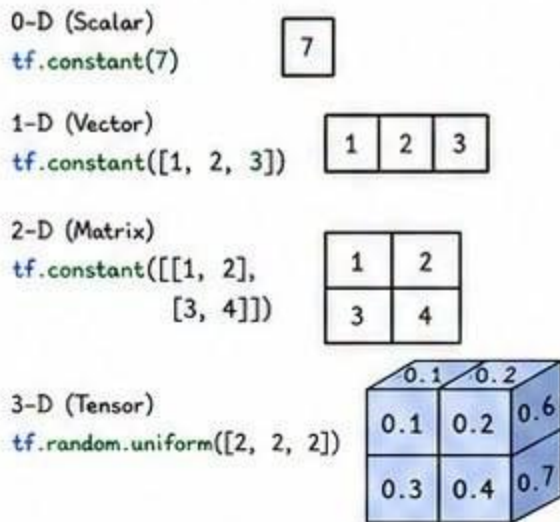
- ✓ RNN is simple but struggles with long-term dependencies.
- ✓ LSTM uses three gates and works well for complex sequences.
- ✓ GRU is a simpler and faster alternative to LSTM.
- ✓ Choose the model based on your data, task and computational resources.
- ✓ All three are powerful for different use cases.

TENSORFLOW

Build & Train Machine Learning Models

Tensors

N-dimensional arrays (like NumPy arrays).



Constants & Variables

Constant (cannot be changed)
`x = tf.constant([1.0, 2.0, 3.0])`

Variable (can be changed)
`v = tf.Variable([1.0, 2.0, 3.0])`
`v.assign([4.0, 5.0, 6.0])`
`print(v.numpy())` # [4. 5. 6.]

Basic Operations

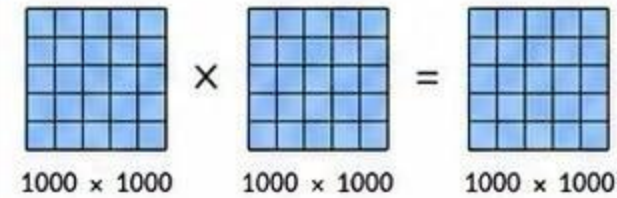
```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

a + b → [5 7 9]
a - b → [-3 -3 -3]
a * b → [4 10 18]
a / b → [0.25 0.4 0.5]

tf.matmul([[1, 2]], [[3], [4]])
→ [[11]]
```

Compute with GPU (if available)

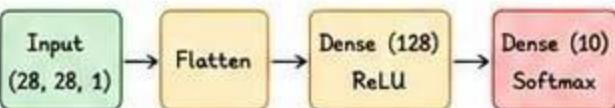
```
with tf.device('/GPU:0'):
    x = tf.random.normal([1000, 1000])
    y = tf.matmul(x, x)
```



Build a Model for MNIST (Multi-class)

Sequential API (Layers stacked in order)

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```



Compile the Model

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

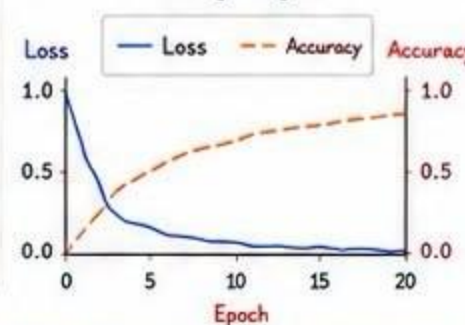
Common Optimizers

- 'adam' (default & powerful)
- 'sgd'
- 'rmsprop'

Train the Model

```
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

Training Progress



Evaluate the Model

```
loss, acc = model.evaluate(X_test, y_test)
print(f"Loss: {loss:.4f}")
print(f"Accuracy: {acc:.4f}")
```

Make Predictions

```
# Predict class probabilities for 5 samples
probs = model.predict(X_test[:5])
print("Probabilities (for 10 classes):")
print(probs)
# Predicted class = index of highest probability
preds = tf.argmax(probs, axis=1)
print("Predicted classes:", preds.numpy())
```

[0.03 0.01 0.05 0.02 0.89 ... 0.00]	→ Class 8
[0.01 0.02 0.02 0.01 0.01 ... 0.00]	→ Class 1
[0.04 0.02 0.03 0.90 0.01 ... 0.00]	→ Class 3
[0.00 0.00 0.01 0.00 0.97 ... 0.00]	→ Class 4
[0.02 0.01 0.88 0.02 0.02 ... 0.00]	→ Class 2

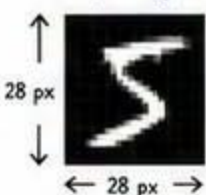
(Each row shows probabilities for 10 classes)

Load & Prepare Data (MNIST)

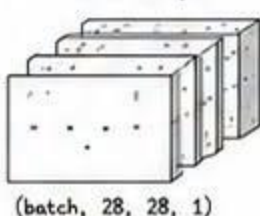
```
(X_train, y_train), (X_test, y_test) = \
    tf.keras.datasets.mnist.load_data()

X_train = X_train / 255.0
X_test = X_test / 255.0
X_train = X_train[..., tf.newaxis] # (60000, 28, 28, 1)
X_test = X_test[..., tf.newaxis] # (10000, 28, 28, 1)
```

Sample Digit



Batch Shape



Callbacks (Improve Training)

```
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=3,
        restore_best_weights=True),
    tf.keras.callbacks.ModelCheckpoint(
        'best_model.keras', save_best_only=True)
]
```

```
model.fit(X_train, y_train, epochs=50,
        validation_split=0.2,
        callbacks=callbacks)
```

Useful Callbacks

- EarlyStopping
- ReduceLROnPlateau
- ModelCheckpoint
- TensorBoard

Save & Load Model

```
# Save the entire model
model.save('my_model.keras')

# Load the model
loaded_model = tf.keras.models
    .load_model('my_model.keras')

# Save weights only
model.save_weights('my_model.weights.h5')

loaded_model.load_weights('my_model.weights.h5')
```

TensorBoard (Visualization)

```
log_dir = "logs/fit/"
tensorboard_cb = tf.keras.callbacks.TensorBoard(
    log_dir=log_dir, histogram_freq=1
)
```

```
model.fit(X_train, y_train,
        epochs=10,
        callbacks=[tensorboard_cb])
```

```
# In terminal, run:
# tensorboard --logdir logs/fit

# Open in browser:
# http://localhost:6006
```



Common Activations

ReLU	<code>tf.nn.relu(x)</code>
Sigmoid	<code>tf.nn.sigmoid(x)</code>
Tanh	<code>tf.nn.tanh(x)</code>
Softmax	<code>tf.nn.softmax(x)</code>
Leaky ReLU	<code>tf.nn.leaky_relu(x)</code>



Common Loss Functions

MSE	<code>'mse'</code>
Binary Crossentropy	<code>'binary_crossentropy'</code>
Categorical Crossentropy	<code>'categorical_crossentropy'</code>
Sparse Categorical Crossentropy	<code>'sparse_categorical_crossentropy'</code>

Useful Utilities

<code>tf.reduce_mean(x)</code>	# Mean
<code>tf.reduce_sum(x)</code>	# Sum
<code>tf.argmax(x, axis=1)</code>	# Argmax
<code>tf.one_hot(y, depth)</code>	# One-hot encode

Random Numbers

<code>tf.random.normal(shape)</code>
<code>tf.random.uniform(shape)</code>
<code>tf.random.set_seed(42)</code>



TensorFlow = Flexible • Scalable • Production-Ready
End-to-End framework for Machine Learning & Deep Learning!

Import TensorFlow
`import tensorflow as tf`

